



UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"
FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Relazione di progetto

JTrash

Settembre 7, 2023

Candidato:

Lorenzo Sabatino, [REDACTED]@studenti.uniroma1.it

Matricola [REDACTED]

Docente:

Prof. Stefano Faralli

Anno Accademico 2022-2023

Indice

1	Introduzione	2
1.1	Informazioni Personali	2
1.2	Obiettivo del Progetto	2
2	Decisioni di Progettazione	2
2.1	Gestione del Profilo Utente	2
2.2	Gestione delle Partite	3
2.3	Interfaccia Grafica e Riproduzione Audio	4
2.4	Architettura MVC e Design Pattern	5
2.5	Utilizzo dei Stream	6
3	Conclusioni	7

1 Introduzione

Il presente documento rappresenta la relazione individuale relativa al progetto *JTrash*, realizzato nell'ambito del corso **Metodologie di Programmazione**. L'obiettivo di questo progetto è di applicare le competenze acquisite durante il corso nell'implementazione di un gioco di carte, conformemente alle specifiche fornite.

1.1 Informazioni Personali

Il sottoscritto, **Lorenzo Sabatino** iscritto al corso di **Metodologie di Programmazione** in modalità di studio **presenza MZ** con numero di matricola [REDACTED], ha sviluppato il progetto *JTrash* rispettando le linee guida e le specifiche indicate. Questa relazione dettaglierà l'implementazione, le decisioni di progettazione, l'uso dei design pattern e altre scelte effettuate per la creazione del gioco.

1.2 Obiettivo del Progetto

L'obiettivo principale di *JTrash* consiste nello sviluppo di un gioco di carte interattivo, che permetta ai giocatori di sfidarsi in partite contro avversari artificiali. Il gioco include funzionalità di gestione del profilo utente, quali la personalizzazione del nickname e dell'avatar, oltre al tracciamento delle partite giocate e dei risultati ottenuti. L'aspirazione è quella di creare un'esperienza coinvolgente e divertente, mantenendo contemporaneamente un'architettura del software e un codice puliti.

2 Decisioni di Progettazione

In questa sezione, verranno descritte le decisioni chiave di progettazione prese durante lo sviluppo del progetto *JTrash*. Saranno esposte le strategie adottate per soddisfare le specifiche fornite e implementare le funzionalità richieste, come la gestione del profilo utente, la gestione delle partite, l'interfaccia grafica, l'uso dei design pattern e altre scelte progettuali rilevanti.

2.1 Gestione del Profilo Utente

Per la gestione del profilo utente, ho adottato un approccio basato su classi e oggetti per garantire la separazione dei dati utente e la loro interazione con il resto del gioco.

L'interfaccia utente per la gestione del profilo è stata implementata attraverso la classe **ProfiloScreen**. Questa schermata mostra al giocatore il suo nickname, le statistiche delle partite e il livello raggiunto. Ho adottato il formato JSON per memorizzare e gestire questi dati. In particolare, ho creato un file chiamato **stats.json** che contiene le informazioni del profilo utente. Per leggere e scrivere i dati in formato JSON, ho utilizzato il pacchetto **json.jar**, che ha semplificato il processo di accesso ai dati e la loro persistenza.

Inoltre, ho voluto arricchire l'esperienza utente offrendo la possibilità di selezionare un avatar personalizzato dal gioco. Ho implementato una scelta tra quattro personaggi inerenti alla storia di *JTrash*, ognuno con voci e immagini

personalizzate. Questa funzionalità ha contribuito a coinvolgere ulteriormente il giocatore nell'atmosfera del gioco e ha aggiunto un elemento di personalizzazione unico al profilo utente.



Figura 1: Schermata di gestione del profilo utente.

2.2 Gestione delle Partite

Nel contesto di *JTrash*, ho scelto di adottare un'architettura model-view-controller (MVC) per una gestione efficiente delle partite (vedi Sezione 2.4). La classe `TrashGame` rappresenta il cuore del modello di gioco, occupandosi principalmente della gestione dei giocatori e del coordinamento delle varie fasi di una partita. In particolare, `TrashGame` è responsabile di inizializzare il mazzo delle carte, distribuirle ai giocatori e avviare i nuovi round.

La classe `Giocatore` è stata progettata per gestire tutte le azioni legate ai giocatori all'interno del gioco di carte. Essa contiene metodi per giocare carte, effettuare mosse strategiche e tenere traccia delle risorse disponibili. I giocatori comunicano con il modello di gioco tramite questa classe, contribuendo così all'interazione dinamica all'interno dell'ambiente di gioco.

Nel controller, ho sviluppato la classe `JTrash` che gestisce i turni di gioco. Utilizzando un ciclo `while(true)`, `JTrash` regola il flusso del gioco, consentendo ai giocatori umani e artificiali di effettuare le proprie mosse in modo sequenziale.

Ogni partita ha inizio quando nella GUI viene scelta una modalità di gioco: "facile" contro un giocatore, "normale" contro due giocatori, "difficile" contro tre giocatori. Solo quando l'utente ha effettuato questa scelta, il gioco viene avviato attraverso la creazione di un nuovo thread utilizzando `SwingWorker`:

```
SwingWorker<Void, Void> worker = new SwingWorker<Void, Void>() {
    @Override
    protected Void doInBackground() throws Exception {
        JTrash.startGame();
        return null;
    }
};
worker.execute();
```

Questo approccio consente di avviare il loop del controller precedentemente descritto e di dare inizio al gioco.

2.3 Interfaccia Grafica e Riproduzione Audio

L'interfaccia grafica di *JTrash* è stata sviluppata utilizzando la libreria Java Swing, con particolare attenzione alla creazione di un'esperienza visiva coinvolgente e in linea con la storia immaginata per il gioco. Questo contesto ha guidato la progettazione dell'interfaccia grafica, dove ogni elemento visivo è stato adattato per rispecchiare uno scenario futuristico.

Ho creato una classe `FuturisticButton` per personalizzare l'aspetto dei pulsanti presenti nell'interfaccia. Questi pulsanti sono stati disegnati con forme e colori che richiamano l'estetica futuristica.

Per garantire una coerenza stilistica anche nelle scritte dei titoli, ho utilizzato la classe `CustomFont`. In particolare, ho impiegato il font `Orbitron.ttf` per dare un aspetto moderno e tecnologico ai titoli e agli elementi di testo presenti nell'interfaccia.

Le immagini presenti in ogni sfondo e quelle dei personaggi sono state selezionate con cura tramite l'uso di intelligenza artificiale per adattarle allo stile richiesto. Questo processo ha contribuito a creare un'atmosfera unica e coinvolgente, permettendo ai giocatori di immergersi completamente nel mondo di *JTrash*.



Figura 2: Esempio dello stile futuristico dell'interfaccia grafica nel menu principale.

Per arricchire ulteriormente l'esperienza di gioco, ho aggiunto suoni personalizzati ai personaggi e alle carte, rendendo le interazioni più dinamiche e coinvolgenti. Inoltre, ho incluso una musica di sottofondo appropriata al tema futuristico del gioco, aggiungendo qualche metodo alla classe `AudioManager`, che contribuisce a creare un'atmosfera avvincente durante le partite.

L'adozione di uno stile futuristico coerente in tutto il progetto ha contribuito a trasmettere l'atmosfera e la narrazione del gioco, offrendo un'esperienza visiva e sonora completa e immersiva ai giocatori.

2.4 Architettura MVC e Design Pattern

Durante lo sviluppo di *JTrash*, ho applicato diversi design pattern al fine di migliorare la struttura del codice e incrementare la sua flessibilità. In particolare, ho adottato il design pattern MVC (Model-View-Controller) per garantire una suddivisione chiara delle responsabilità e una separazione efficace tra le diverse componenti del gioco.

All'interno dell'architettura MVC, ho organizzato il codice in modo da separare le varie funzionalità del gioco. Nel modello (Model), ho inserito le classi che si occupano della logica di gioco, tra cui **TrashGame**, **Giocatore** e altre classi correlate. La vista (View) è composta dalle classi responsabili di presentare le informazioni all'utente, come la classe **BoardPanel**, **PlayerPanel**, **ProfiloScreen** e gli elementi grafici. Infine, tramite il controller, ho gestito la comunicazione tra il modello e la vista, garantendo l'interazione coerente tra le diverse componenti del gioco.

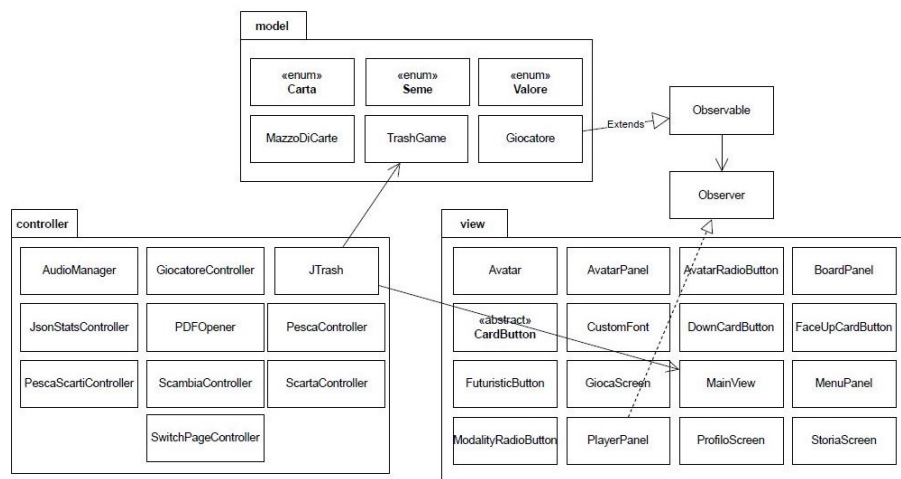


Figura 3: Illustrazione della divisione con MVC in *JTrash*.

In aggiunta al design pattern MVC, ho utilizzato il pattern Observer/Observable per gestire una comunicazione più veloce e reattiva tra il modello e la vista. Ho reso la classe **Giocatore** (Player) un oggetto osservabile e la classe **PlayerPanel** un osservatore. In questo modo, quando avviene un evento rilevante, come un giocatore che pesca una carta o effettua una mossa, il modello notifica la vista attraverso il pattern Observer. Di conseguenza, **PlayerPanel** può essere aggiornato in tempo reale per riflettere il cambiamento e fornire un'esperienza interattiva all'utente.

Nel mio approccio, ho implementato un metodo di notifica personalizzato all'interno della classe **Giocatore**, che permette di inviare notifiche specifiche alla vista. Ecco un esempio semplificato del metodo di notifica:

```
private void notifica(String message, Object... obj) {  
    setChanged();  
  
    List<Object> notificationList = new ArrayList<>();  
    notificationList.add(message);
```

```

        notificationList.addAll(Arrays.asList(obj));

        notifyObservers(notificationList);
    }

```

In `PlayerPanel`, l'Observer riceve la notifica inviata dal modello e, tramite il messaggio specificato, esegue le azioni corrispondenti. Ad esempio, può gestire eventi come la vittoria di un giocatore, la pesca di una carta, lo scambio di carte e altro ancora. Questo approccio consente una gestione dinamica e reattiva dell'interfaccia utente, assicurando che l'utente riceva aggiornamenti in tempo reale sullo stato del gioco.

L'utilizzo combinato del design pattern MVC e Observer/Observable ha migliorato significativamente la struttura e la reattività del codice, consentendo una gestione efficiente delle comunicazioni tra il modello e la vista e offrendo un'esperienza di gioco più coinvolgente.

Infine ho adottato il design pattern Singleton in alcune parti cruciali del codice, tra cui la classe `TrashGame` e il frame `MainView`. Questa scelta è stata guidata dall'obiettivo di garantire l'accesso controllato e unico a queste istanze, al fine di mantenere coerenza e consistenza nell'intero sistema.

Nel caso della classe `TrashGame`, ho implementato il Singleton per assicurare che vi sia sempre una sola istanza attiva del gioco in tutto il sistema. Questo è particolarmente utile per evitare conflitti o inconsistenze quando diversi componenti del gioco tentano di interagire con il modello di gioco contemporaneamente. L'uso del Singleton garantisce che il `TrashGame` sia sempre accessibile e condiviso tra le varie parti del codice, facilitando la gestione delle partite, delle mosse e delle interazioni tra i giocatori.

Per quanto riguarda il frame `MainView`, ho adottato il Singleton per garantire che vi sia un'unica istanza del frame principale dell'applicazione. Ciò permette di evitare la creazione di più istanze del frame, che potrebbero causare problemi di gestione delle finestre e interferire con l'esperienza utente. Inoltre, utilizzando il Singleton, è possibile accedere in modo coerente alle funzionalità del frame principale da diverse parti del codice, semplificando la gestione dell'interfaccia utente.

2.5 Utilizzo dei Stream

Durante lo sviluppo di *JTrash*, ho sfruttato la potenza degli stream di Java per semplificare e migliorare l'efficienza di alcune operazioni all'interno del codice.

Un esempio di utilizzo degli stream si trova nella classe `Giocatore`, nel metodo `isCarteGirate()`. Questo metodo verifica se tutte le carte dello stato attuale delle carte del giocatore sono girate, restituendo `true` se lo sono e `false` altrimenti. Per ottenere questo risultato, ho utilizzato uno stream per accedere ai valori di stato delle carte (`statoDelleCarte`) e ho applicato l'operazione `allMatch(Boolean::booleanValue)` per verificare se tutti i valori sono `true`. L'uso dello stream rende il codice più compatto ed elegante, semplificando la logica di verifica dello stato delle carte.

Nella classe `PlayerPanel`, ho utilizzato uno stream nel metodo `setEnabledButton(boolean enabledValue)`. Questo metodo imposta lo stato di abilitazione dei pulsanti sul pannello. Gli stream vengono utilizzati per iterare attraverso la collezione

di bottoni delle carte (`cards`) e abilitarli o disabilitarli in base al valore fornito (`enabledValue`).

La necessità di gestire la disabilitazione dei pulsanti delle carte è emersa dall'approccio di progettazione che ho adottato. Al fine di mantenere un codice pulito e facilmente gestibile, ho esteso la classe `JButton` creando la classe `CardButton`. Questa estensione mi ha permesso di considerare tutte le carte del gioco, comprese quelle degli avversari, come istanze di `CardButton`. Tuttavia, per garantire un'esperienza di gioco coerente, ho dovuto gestire la disabilitazione dei pulsanti degli avversari in modo da impedire l'interazione con le loro carte durante i turni degli altri giocatori.

L'utilizzo di uno stream nel metodo `setEnabledButton` ha semplificato il processo di abilitazione e disabilitazione dei pulsanti delle carte, fornendo una soluzione elegante e efficiente per gestire questa operazione su tutti gli elementi della collezione. Questo ha contribuito a una maggiore chiarezza del codice e a una gestione uniforme dei pulsanti delle carte all'interno del gioco.

3 Conclusioni

Il progetto *JTrash* rappresenta un risultato tangibile del mio impegno e della mia passione nello sviluppo di un gioco di carte interattivo. Le principali caratteristiche implementate e le sfide affrontate hanno contribuito a creare un prodotto completo e coinvolgente. In questa sezione, riassumerò le caratteristiche chiave del progetto e condividerò le mie riflessioni finali sull'esperienza di sviluppo.

Le principali caratteristiche di *JTrash* includono la gestione del profilo utente, che consente ai giocatori di personalizzare il loro avatar e monitorare le statistiche delle partite. L'uso del pattern MVC ha garantito una struttura chiara e modulare del codice, facilitando la gestione delle interazioni tra modello, vista e controller. Inoltre, l'implementazione di design pattern come Observer/Observable e l'utilizzo degli stream hanno migliorato l'efficienza e la leggibilità del codice.

L'interfaccia grafica ha giocato un ruolo fondamentale nell'esperienza di gioco, grazie all'uso di immagini generate tramite intelligenza artificiale, suoni personalizzati e un'atmosfera futuristica. L'integrazione di elementi visivi e sonori coerenti ha contribuito a creare un'atmosfera coinvolgente e immersiva per i giocatori.

Durante l'esperienza di sviluppo, ho affrontato sfide tecniche e concettuali, ma ogni sfida è stata un'opportunità di apprendimento. L'approccio strategico alla risoluzione dei problemi, insieme all'uso efficace di risorse come documentazione e comunità online, ha giocato un ruolo cruciale nel superare le difficoltà incontrate. Sono particolarmente soddisfatto dell'adozione di design pattern e dell'implementazione di funzionalità avanzate, come la gestione dei turni e l'interazione tra giocatori artificiali.

Tra gli aspetti più gratificanti di questo progetto, c'è stata la possibilità di vedere prendere vita un'idea attraverso il processo di sviluppo. La sensazione di realizzazione nel vedere il gioco funzionante e l'interfaccia utente ben progettata è stata estremamente gratificante. Inoltre, l'esperienza ha rafforzato la mia capacità di pianificare, progettare e implementare un progetto software completo.